Jenkins ⭐ ⭐ ⭐ — Interview-Ready Q&A

1. What is Jenkins?
Jenkins is an open-source automation server mainly used for CI/CD.
It can automatically build, test, package, and deploy applications whenever code changes are pushed to a source-code repository.
In DevOps, Jenkins helps automate the software delivery process and reduces manual work.
Simple ga gurthupettuko:
Jenkins = CI/CD automation tool.
Code → Build → Test → Deploy
Eng: Jenkins is an open-source automation server used to implement CI/CD pipelines and automate software delivery.

---

2. What is CI?
CI stands for **Continuous Integration**.
Developers frequently push their code changes to a shared repository, and an automated pipeline builds and tests those changes.
The main goal is to identify integration or code issues early instead of finding them at the end of development.
Simple ga gurthupettuko:
CI = Code frequently integrate + automatically build/test.
Eng: Continuous Integration is the practice of frequently integrating code changes and automatically building and testing them.

---

3. What is CD?
CD can mean **Continuous Delivery** or **Continuous Deployment**, depending on the context.
Continuous Delivery means the application is automatically prepared for release, while Continuous Deployment automatically deploys validated changes to the target environment.
Simple ga gurthupettuko:
Delivery = Deploy ki ready
Deployment = Automatically deploy
Eng: Continuous Delivery keeps software ready for release, while Continuous Deployment automatically releases validated changes.

---

4. What is a Jenkins Job?
A Jenkins Job is a configured task that Jenkins executes.
It can perform activities such as checking out code, running Maven builds, executing tests, creating Docker images, or deploying applications.
Simple ga gurthupettuko:
Jenkins Job = Jenkins execute cheyyalsina task.
Eng: A Jenkins Job is a configured task used to automate activities such as build, test, and deployment.

---

5. What is a Jenkins Pipeline?
A Jenkins Pipeline is a sequence of automated steps used to implement a CI/CD process.
It defines stages such as checkout, build, test, quality scan, package, and deployment.
For example:
Checkout → Build → Test → SonarQube → Docker Build → Deploy
Simple ga gurthupettuko:
Pipeline = Complete CI/CD flow.
Eng: A Jenkins Pipeline defines the automated stages required to build, test, and deploy an application.

---

6. What is Jenkinsfile?
A Jenkinsfile is a text file that defines a Jenkins Pipeline as code.
It is usually stored along with the application source code in Git, which means the pipeline configuration can also be version controlled.
Simple ga gurthupettuko:
Jenkinsfile = Pipeline code.
Pipeline configuration ni Jenkins UI lo manually maintain cheyyakunda Git lo store cheyyachu.
Eng: A Jenkinsfile defines a Jenkins Pipeline as code and is commonly stored in the source-code repository.

---

7. What is the difference between Declarative and Scripted Pipeline?
Declarative Pipeline uses a structured and predefined syntax that makes pipelines easier to read and maintain.
Scripted Pipeline provides more flexibility because it is based on Groovy scripting and is useful when complex logic is required.
For most standard CI/CD pipelines, Declarative Pipeline is easier to maintain.
Simple ga gurthupettuko:
Declarative = Simple + Structured
Scripted = Flexible + More coding
Eng: Declarative pipelines provide a structured syntax, while Scripted pipelines provide greater flexibility for complex logic.

---

8. What are stages in Jenkins Pipeline?
Stages represent logical sections of a pipeline.
Common stages include **Checkout, Build, Test, Code Quality, Package, and Deploy**.
Stages make the pipeline easier to understand and help identify where a failure occurred.
Simple ga gurthupettuko:
Stage = Pipeline lo oka major step/section.
Eng: Stages divide a Jenkins Pipeline into logical sections such as build, test, and deployment.

9. What is an Agent in Jenkins?
An Agent is a machine or execution environment where Jenkins performs pipeline tasks.
The Jenkins controller can assign work to agents, allowing builds and deployments to run on different machines.
For example, one agent can be configured for Java builds and another for Docker-related tasks.
Simple ga gurthupettuko:
Controller = Pipeline manage
Agent = Actual commands execute
Eng: A Jenkins agent is an execution environment where pipeline tasks are performed.

---

10. What is Jenkins Controller?
The Jenkins Controller is the central Jenkins component responsible for managing the Jenkins environment.
It handles pipeline configuration, scheduling, credentials, plugins, and assigning work to agents.
Build workloads should generally be executed on agents rather than putting all workload directly on the controller.
Simple ga gurthupettuko:
Controller = Jenkins brain/manager
Agent = Work execution machine
Eng: The Jenkins Controller manages Jenkins configuration and schedules and delegates work to agents.

---

11. What are Jenkins Plugins?
Plugins extend Jenkins functionality and allow Jenkins to integrate with external tools.
For example, plugins can integrate Jenkins with Git, Maven, Docker, AWS, Kubernetes, SonarQube, and notification systems.
Simple ga gurthupettuko:
Plugin = Jenkins ki additional capability.
Eng: Jenkins plugins extend its functionality and enable integration with external tools and platforms.

---

12. What are Jenkins Credentials?
Jenkins Credentials provide a secure way to store sensitive authentication information required by pipelines.
Examples include usernames/passwords, SSH keys, API tokens, and cloud credentials.
Instead of hard-coding credentials inside a Jenkinsfile, we reference credentials securely from Jenkins.
Simple ga gurthupettuko:
Credentials = Passwords/tokens/keys secure ga manage cheyyadaniki.
Eng: Jenkins Credentials securely store authentication information used by jobs and pipelines.

---

13. How do you integrate Jenkins with GitHub?
Jenkins can connect to GitHub repositories and automatically retrieve source code.
A GitHub webhook can notify Jenkins when a push or Pull Request event occurs, allowing Jenkins to trigger the required pipeline.
Typical flow:
Developer → GitHub → Webhook → Jenkins → Build/Test → Deploy
Simple ga gurthupettuko:
GitHub event → Jenkins trigger → Pipeline
Eng: Jenkins can integrate with GitHub using repository configuration and webhooks to trigger CI/CD pipelines.

---

14. What is a Jenkins Webhook?
A webhook allows an external system such as GitHub to notify Jenkins about repository events.
For example, when code is pushed to GitHub, GitHub sends an HTTP request to Jenkins, and Jenkins can start the configured pipeline.
This is generally more efficient than continuously polling the repository.
Simple ga gurthupettuko:
GitHub Push → Webhook → Jenkins
Eng: A webhook allows GitHub to notify Jenkins immediately when repository events occur and trigger automation.

---

15. What is Jenkins Poll SCM?
Poll SCM means Jenkins periodically checks the source-code repository for changes.
If Jenkins detects a new change, it can trigger the configured job.
Webhook-based triggering is often preferred because it can start the pipeline immediately instead of waiting for the next polling interval.
Simple ga gurthupettuko:
Poll SCM = Jenkins repeatedly checks Git for changes.
Eng: Poll SCM periodically checks the source repository and triggers a job when changes are detected.

---

16. What is the difference between Webhook and Poll SCM?
With a webhook, the source-control system actively notifies Jenkins when an event occurs.
With Poll SCM, Jenkins repeatedly checks the repository to find whether changes are available.
Webhook is generally more event-driven and immediate, while Poll SCM introduces periodic checking.
Simple ga gurthupettuko:
Webhook = GitHub tells Jenkins
Poll SCM = Jenkins asks GitHub
Eng: A webhook pushes an event notification to Jenkins, while Poll SCM makes Jenkins periodically check for changes.

17. How do you pass parameters to a Jenkins job?
A Jenkins job can be configured as a parameterized job.
Parameters can include values such as environment name, application version, branch name, or deployment option.
These parameters can then be used inside the pipeline to control its behavior.
Simple ga gurthupettuko:
Parameter = Pipeline run time lo value pass cheyyadam.
Eng: Jenkins parameters allow users or automation to provide values that control how a job executes.

---

18. What is an environment variable in Jenkins?
An environment variable is a key-value pair made available to a build or pipeline.
It can store non-sensitive configuration information such as environment names, paths, or application settings.
Sensitive values should be handled using Jenkins Credentials rather than plain environment variables.
Simple ga gurthupettuko:
Environment Variable = Runtime configuration value.
Eng: Environment variables provide configuration values to Jenkins jobs during execution.

---

19. How do you archive artifacts in Jenkins?
After a build, Jenkins can archive generated files as build artifacts.
For example, a Maven build may generate a .jar file, which can be archived so it can be downloaded or used by later stages or jobs.
Simple ga gurthupettuko:
Build output → Artifact → Archive
Eng: Jenkins can archive build artifacts so generated files can be stored and reused after the build.

---

20. What is an artifact in CI/CD?
An artifact is a packaged output generated during the build process.
Examples include JAR files, WAR files, ZIP packages, Docker images, and other deployable packages.
The artifact created during CI can be promoted through environments without rebuilding the source each time.
Simple ga gurthupettuko:
Artifact = Deploy cheyyagalige build output.
Eng: An artifact is a versioned output produced by a build that can be stored and deployed.

---

21. How do you handle build failures in Jenkins?
First, I check the failed stage and Jenkins console logs to identify the actual error.
Then I determine whether the issue is related to source code, dependencies, environment configuration, credentials, network connectivity, or infrastructure.
After fixing the root cause, I rerun the pipeline and verify that the complete flow succeeds.
Simple ga gurthupettuko:
Failed Stage → Logs → Root Cause → Fix → Rerun → Verify
Eng: I troubleshoot Jenkins failures by checking the failed stage and logs, identifying the root cause, fixing it, and rerunning the pipeline.

---

22. How do you troubleshoot a Jenkins pipeline that is stuck?
First, I check which stage or step is waiting and inspect the console output.
Then I check whether the required agent is available, whether another build is holding a resource, whether there is a network issue, or whether the pipeline is waiting for manual input.
I also check Jenkins system logs if the problem is not visible in the pipeline output.
Simple ga gurthupettuko:
Stuck Stage → Agent → Resource → Input → Logs
Eng: I troubleshoot a stuck pipeline by checking the running stage, agent availability, resource contention, manual approvals, and Jenkins logs.

---

23. How do you manage Jenkins credentials securely?
I store credentials in the Jenkins Credentials Store instead of putting passwords or tokens directly in Jenkinsfiles.
Pipelines access the required credential through Jenkins' credential mechanisms, and access should follow the principle of least privilege.
For cloud deployments, short-lived authentication mechanisms should be preferred where supported.
Simple ga gurthupettuko:
No hard-code → Credentials Store → Least Privilege
Eng: I manage secrets using Jenkins Credentials and avoid hard-coding sensitive values in pipeline code.

---

24. What is Jenkins Build Trigger?
A build trigger defines what event or condition starts a Jenkins job.
Common triggers include GitHub webhooks, Poll SCM, scheduled builds, upstream jobs, and manual execution.
For example, a GitHub push can trigger the CI pipeline automatically.
Simple ga gurthupettuko:
Trigger = Pipeline/job start cheyyadaniki reason.
Eng: A Jenkins build trigger defines the event or condition that starts a job.

25. How do you deploy an application using Jenkins?
First, Jenkins checks out the application source code from Git.
Then it builds and tests the application, performs required quality/security checks, creates the deployable artifact or container image, and deploys it to the target environment.
After deployment, we should verify application health and monitor the deployment.
Simple ga gurthupettuko:
Checkout → Build → Test → Scan → Package → Deploy → Verify
Eng: Jenkins can automate the complete deployment process from source checkout through build, testing, packaging, deployment, and verification.

---

26. How do you implement approval before Production deployment?
We can configure a manual approval step in the Jenkins pipeline before the production deployment stage.
The pipeline pauses and waits for an authorized person to approve the deployment.
This provides an additional control so that successful CI does not automatically mean immediate Production deployment.
Simple ga gurthupettuko:
CI Pass → Approval → Production Deploy
Eng: I can add a manual approval gate before the Production deployment stage to ensure authorized approval.

---

27. How do you run Jenkins builds on different environments?
We can use different Jenkins agents or labels for different environments or workloads.
For example, one agent can be configured for Linux builds, another for Docker, and another for specialized deployment tasks.
The pipeline can select the required agent based on the stage requirements.
Simple ga gurthupettuko:
Different Work → Different Agents
Eng: Jenkins can use labeled agents to execute different pipeline stages in suitable execution environments.

---

28. How do you make Jenkins highly available?
Jenkins Controller itself requires careful architecture because the controller is stateful.
In practice, we protect the Jenkins environment using backups, durable storage, configuration-as-code where appropriate, redundant infrastructure around dependencies, and reliable agents. Jenkins workloads can be distributed across multiple agents.
For critical environments, recovery procedures and regular backups should be tested rather than assuming availability automatically.
Simple ga gurthupettuko:
Backup + Durable Storage + Multiple Agents + Recovery Plan
Eng: I improve Jenkins resilience through backups, durable storage, distributed agents, configuration management, and tested recovery procedures.

---

29. What is Jenkins Shared Library?
A Shared Library allows common pipeline code and reusable functions to be maintained separately and reused across multiple Jenkins pipelines.
Instead of duplicating the same deployment or notification logic in every Jenkinsfile, we can create reusable functions.
This improves maintainability and standardization across projects.
Simple ga gurthupettuko:
Shared Library = Common pipeline code ni once create chesi multiple projects lo reuse cheyyadam.
Eng: Jenkins Shared Libraries provide reusable pipeline code that can be shared across multiple projects.

---

30. What is the typical Jenkins CI/CD pipeline?
A typical pipeline starts when a developer pushes code to GitHub or another source repository.
Jenkins checks out the code, builds the application, runs tests, performs code-quality and security checks, creates an artifact or container image, and deploys it to the required environment.
Production deployment may include an approval step depending on the organization's process.
Simple ga gurthupettuko:
GitHub → Jenkins → Checkout → Build → Test → Scan → Artifact → Deploy → Monitor
Eng: A typical Jenkins pipeline automates source checkout, build, testing, quality checks, packaging, deployment, and post-deployment verification.